

論理生命学 課題 石井先生担当分

学籍番号 6930377841 南川健志郎

2025 年 8 月 2 日

レポート課題 5: DQN の実装

近年, 強化学習の近似 TD 法において, 柔軟な近似性能をもつ深層学習を適用した深層強化学習の研究が盛んにおこなわれている. そこで強化学習のなかでも基本的なアルゴリズムである Deep Q-Network(DQN) アルゴリズムの実装を行い, 強化学習手法を高次元な問題へ適用する.

1 問題設定: 数字の昇順ソート

N 桁の数字を昇順にソートする問題を考える. i 桁目の数字を $x_i \in \{0, \dots, 9\}$ として, 以下に各時刻 t における変数を定義する:

観測 s_t

N 桁の数字 $x_0x_1 \dots x_{N-1}$

行動 a_t

$i+1$ ($i = 0, \dots, N-2$) 番目の隣接する 2 つの数字 x_i と x_{i+1} を交換する操作. 確率 1 で a_t の結果に対応する観測 s_{t+1} を得るものとする.

報酬 r_t

数列を昇順にソートできた時点で報酬 1 を, それ以外の場合は, 小さな負の報酬を与える. 具体的には, 最悪の場合に必要な行動数である $N(N-1)/2$ で -1 を割った値とする.

2 アルゴリズム

上記の問題設定における状態の総数は 10^N となり, 表形式の Q 学習では技術的に困難になる. そこで, Q 関数をニューラルネットワークによる非線形関数近似手法を用いたアプローチを考える. まず Q 関数 $Q(s, a)$ がパラメータ θ を用いて表されていて, 近似された関数を $Q_\theta(s, a)$ で表されるとする. また $Q_\theta(s, a)$ と同一のネットワークアーキテクチャをもつ関数 $\hat{Q}_\theta(s, a)$ (ターゲットネットワーク) を用意する. 以下に DQN 法のアルゴリズムを示す.

Algorithm 1 Deep Q-Network (DQN) Algorithm

```
1: Initialize parameters  $\theta$ , target parameters  $\hat{\theta} \leftarrow \theta$ , replay buffer  $P$ , and iteration counter  $i \leftarrow 0$ 
2: for each episode do
3:   Initialize  $t = 0$ , observe  $s_t$ 
4:   while  $s_{t+1}$  is not terminal do
5:     Choose action  $a_t$  using  $\epsilon$ -greedy strategy:
6:     With probability  $1 - \epsilon$ , choose  $a_t = \arg \max_a Q_\theta(s_t, a)$ ; otherwise choose random action
7:     Execute action  $a_t$ , observe next state  $s_{t+1}$  and reward  $r_t$ 
8:     Store transition  $b_t = (s_t, a_t, r_t, s_{t+1})$  into replay buffer  $P$ 
9:     if  $A < i$  then
10:      Update  $\theta$ 
11:    end if
12:    if  $i \bmod B = 0$  then
13:       $\bar{\theta} \leftarrow \theta$ 
14:    end if
15:     $i \leftarrow i + 1, s_t \leftarrow s_{t+1}, t \leftarrow t + 1$ 
16:  end while
17: end for
```

3 学習の安定化

DQN 法の安定化手法について説明する.

3.1 Fixed Target Q-network

DQN アルゴリズムでは全ての s_t, a_t に対する値 $Q(s_t, a_t)$ に関して,

$$y_t = \begin{cases} r_t & s_{t+1} \text{が終端状態} \\ r_t + \max_{a'} \hat{Q}_{\bar{\theta}}(s_{t+1}, a') & \text{otherwise} \end{cases} \quad (1)$$

に平均的に近づくように学習が行われる. ここで, 用いられている Q 関数の値はターゲットネットワーク $\hat{Q}_{\bar{\theta}}$ の出力である. 主たるネットワークである $Q_\theta(s, a)$ の古いパラメータ $\bar{\theta}$ を用いたターゲットネットワークの出力を用いることが正則化として働き, 学習が安定する. このような条件のもとで $Q_\theta(s, a)$ の学習を行う. 具体的には, 各時刻 t_i における誤差関数として二乗誤差

$$L_\theta = \frac{1}{2} (y_{t_i} - Q_\theta(s_{t_i}, a_{t_i}))^2 \quad (2)$$

が定義される. ネットワーク $Q_\theta(s, a)$ の学習はこの L_θ の θ に関する勾配

$$\nabla_\theta L_\theta = -(\text{target} - Q_\theta(s_{t_i}, a_{t_i})) \nabla_\theta Q_\theta(s_{t_i}, a_{t_i}) \quad (3)$$

を用いた確率的勾配降下法によって最適化される.

3.2 Experience Replay

従来の Q 学習のように 1 ステップごとにその時刻の内容 (experience) を学習するのではなく、キューに各時刻の内容を保存しておき、キューから内容をランダムに取り出して (replay)、ニューラルネットワークを学習する工夫である。すなわち、

$$L_{\theta} = \mathbb{E}_{b_{t_i} \sim U(P)} \left[\frac{1}{2} (y_{t_i} - Q_{\theta}(s_{t_i}, a_{t_i}))^2 \right] \quad (4)$$

ここで、 $b_{t_i} \sim U(P)$ はキュー P からランダムにサンプルを取り出す操作を表す。各ステップごとにその時刻の内容を学習すると、時間的に相関が高いサンプルばかりになり、バイアスが生じる。ランダムに選ばれたサンプル集合によるミニバッチ学習を行うことで、バイアスの発生を抑制することができる。

3.3 Huber 関数

DQN アルゴリズムでは、実際に誤差関数として二乗誤差ではなく Huber 関数を使用する工夫がなされている。Huber 関数は誤差が -1 から 1 の間は二乗誤差の値になり、-1 より小さいか 1 より大きいときには誤差の絶対値を取る関数である。誤差が大きい場合に二乗誤差を使用すると、誤差関数の出力が大きくなりすぎて学習が安定しづらくなるため、Huber 関数によって安定化を図る。

4 実装

DQN アルゴリズムを用いて 5 桁の数字のソートを行った。プログラムは env.py, agent.py, replaybuffer.py, main.py の 4 つからなる。プログラムの詳細な説明は避け、重要な部分を説明する。具体的なコードは添付のプログラムを参照されたい。

4.1 Env class

env.py では環境を表す SortEnv class を定義している。ここでは数列の生成、入れ替え、報酬計算、描画の機能を実装した。数列は以下の reset 関数によって初期化・生成される。ここで、数列は reset 毎に重複がないかつ一様に選んだ。

Listing 1 reset 関数

```
1 def reset(self):
2     self.state = torch.randperm(10, device=self.device)[:self.N].float()
3     self.steps = 0
4     return self.state.clone()
```

step 関数では引数として action を受け取り、action の数値に応じて状態を入れ替える。reward 関数では状態に応じて 1 章で示した報酬 r_t を返すように設計した。

4.2 Agent class

agent.py では, DQN アルゴリズムを実行する DQNAgent class を定義している. agent class には, ϵ のスケジューリング, ϵ -greedy 法に基づく action の選択, 経験の格納, Q ネットワークの更新, ターゲットネットワークの更新の機能を実装した. ϵ は初期値を 1.0 とし, 指数的減衰により指定の最小値まで小さくする. action の選択では ϵ -greedy 法に基づき, 確率 ϵ でランダムに, $1 - \epsilon$ で Q 値が最大になるような action を選択している. このとき, 以下のように実際に Q ネットワークに state を入力して, 最大となる action を求めている.

Listing 2 select_action 関数

```
1 def select_action(self, state):
2     self.steps_done += 1
3     if random.random() < self.epsilon():
4         return random.randint(0, self.action_dim - 1)
5     with torch.no_grad():
6         state = state.unsqueeze(0).to(self.device)
7         q_values = self.q_net(state)
8         return q_values.argmax(dim=1).item()
```

経験の格納では, 後述する ReplayBuffer クラスのメソッドを用いて $s_t, a_t, r_t, s_{t+1}, done$ の情報をバッファに格納する. Q ネットワークの更新は, 以下のように replaybuffer からサンプルを行い, $Q_\theta, target$ を計算, その差分を Huber Loss によって計算を行い, Adam によって最適化計算をすることで更新する.

Listing 3 update 関数

```
1 def update(self):
2     if len(self.buffer) < self.batch_size:
3         return
4
5     states, actions, rewards, next_states, dones = self.buffer.sample(self.
        batch_size)
6     actions = actions.unsqueeze(1)
7     rewards = rewards.unsqueeze(1)
8     dones = dones.unsqueeze(1)
9
10    q_values = self.q_net(states).gather(1, actions)
11
12    with torch.no_grad():
13        next_q = self.target_net(next_states).max(1, keepdim=True)[0]
14        targets = rewards + (1 - dones) * next_q
15
16    loss = nn.SmoothL1Loss()(q_values, targets)
17    self.optimizer.zero_grad()
18    loss.backward()
19    self.optimizer.step()
```

4.3 replay buffer class

replaybuffer.py では ReplayBuffer class を定義し、バッファへの挿入及びサンプルの機能を実装した。初期化時に buffer のリストを定義し、挿入時は指定のバッファサイズに応じて挿入し、サンプル時はランダムにサンプルしたバッチを返すように設計した。

4.4 main 関数

main.py ではこれまで説明した class を用いて学習を行う main 関数を定義し、学習の実行、学習曲線の描画の機能を実装した。今回は 5 回の試行を別々の seed 値で行い、学習がある seed 値に依存していないかを確認した。学習は train_dqn 関数を通して行われる。学習時は以下のループを回すことで行動の選択、状態、報酬の取得、バッファへの格納、モデルの更新を行っている。

Listing 4 train_dqn 関数内のループ

```
1 for episode in range(num_episodes):
2     s = env.reset()
3     total_reward = 0.0
4     done = False
5
6     while not done:
7         a = agent.select_action(s)
8         s_next, r, done = env.step(a)
9
10        agent.store(s, a, r, s_next, done)
11        if start_step < i:
12            agent.update()
13        if i%sync_freq == 0:
14            agent.sync_target()
15
16        s = s_next
17        total_reward += r
18        i += 1
19        #t += 1
20
21    returns.append(total_reward)
22    time_step.append(i)
23
24    if episode % 10 == 0:
25        print(f"[Episode {episode}] Return: {total_reward:.2f},  $\epsilon$ 
            = {agent.epsilon():.3f}")
```

4.5 結果

図 1 に 5 回の試行における学習曲線を示す。薄い線は実際の累積報酬を、濃い線はその移動平均を取ったものを表す。学習を開始する時刻は 1000 ステップとし、ターゲットネットワークは 100

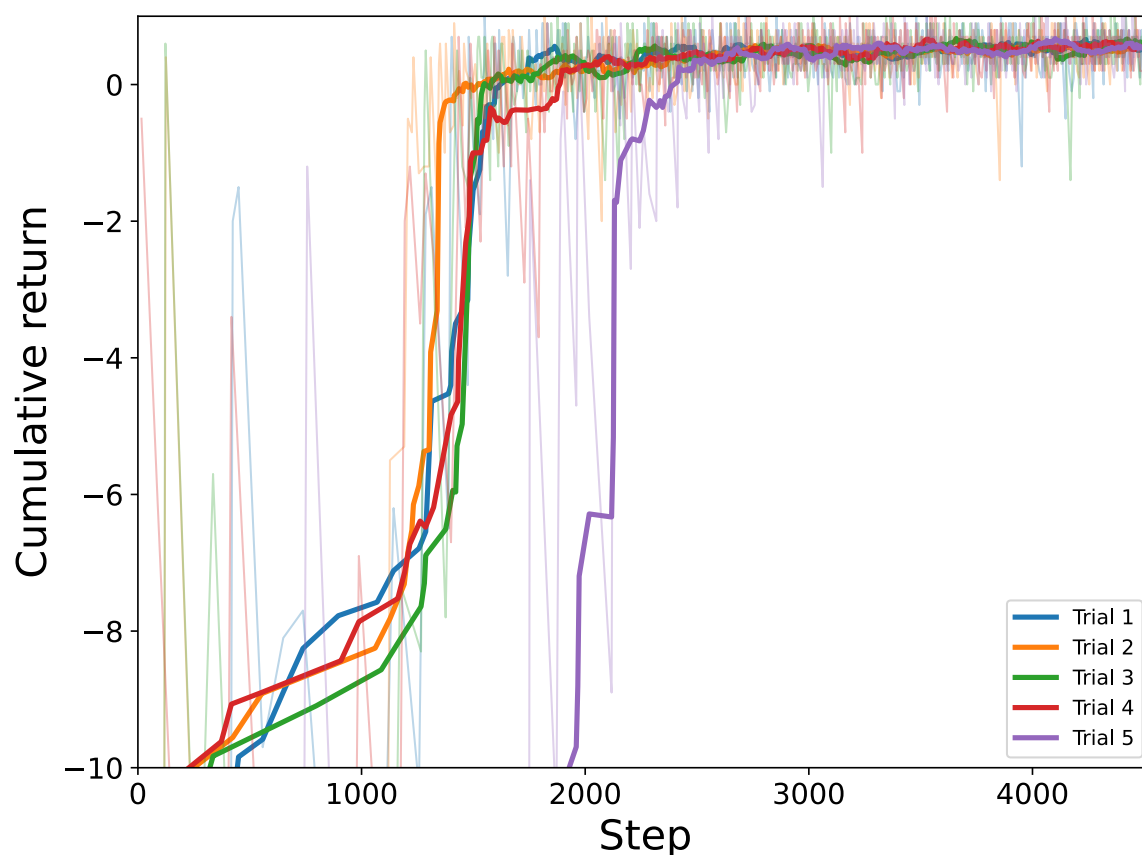


図1 5回の試行における学習曲線

ステップ毎に同期を取る。この図から様々な数列に対してソーティングできる方策が学習できていることが確認できる。実際の sort の例をいくつか示す。この結果から数字の大小を用いて action が選択できていることが確認できる。

Example 1

Step 0: [9.0, 8.0, 4.0, 7.0, 5.0]
 Step 1: [9.0, 4.0, 8.0, 7.0, 5.0]
 Step 2: [9.0, 4.0, 8.0, 5.0, 7.0]
 Step 3: [4.0, 9.0, 8.0, 5.0, 7.0]
 Step 4: [4.0, 8.0, 9.0, 5.0, 7.0]
 Step 5: [4.0, 8.0, 5.0, 9.0, 7.0]
 Step 6: [4.0, 8.0, 5.0, 7.0, 9.0]
 Step 7: [4.0, 5.0, 8.0, 7.0, 9.0]
 Step 8: [4.0, 5.0, 7.0, 8.0, 9.0]

Example 2

Step 0: [4.0, 1.0, 7.0, 9.0, 2.0]
 Step 1: [4.0, 1.0, 7.0, 2.0, 9.0]
 Step 2: [1.0, 4.0, 7.0, 2.0, 9.0]
 Step 3: [1.0, 4.0, 2.0, 7.0, 9.0]
 Step 4: [1.0, 2.0, 4.0, 7.0, 9.0]

Example 3

Step 0: [7.0, 3.0, 1.0, 8.0, 2.0]
 Step 1: [3.0, 7.0, 1.0, 8.0, 2.0]
 Step 2: [3.0, 7.0, 1.0, 2.0, 8.0]
 Step 3: [3.0, 1.0, 7.0, 2.0, 8.0]
 Step 4: [1.0, 3.0, 7.0, 2.0, 8.0]
 Step 5: [1.0, 3.0, 2.0, 7.0, 8.0]
 Step 6: [1.0, 2.0, 3.0, 7.0, 8.0]

5 MNIST を用いたソーティング

先程の課題では状態がそれぞれのスカラー値であった。本章では MNIST の手書き数字を用いてソーティングが可能か検証する。実装として, env.py, agent.py にそれぞれ MNIST に対応

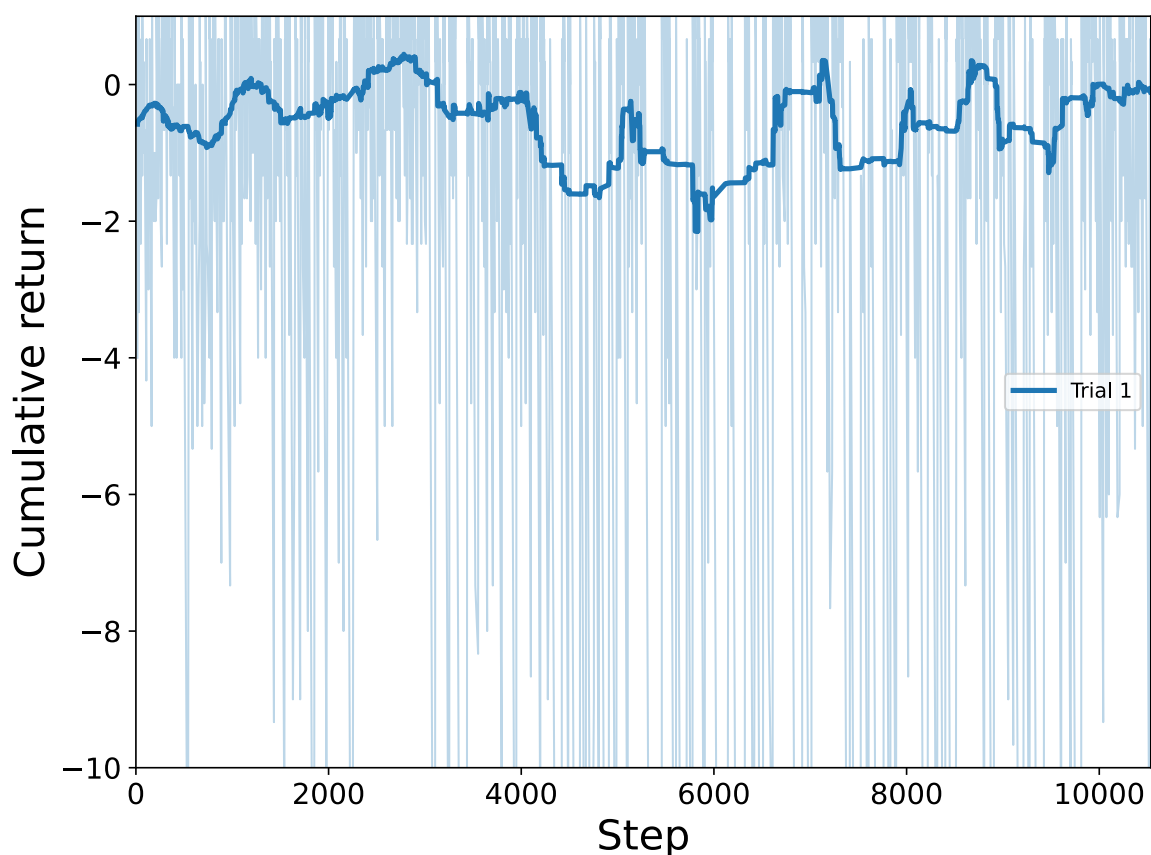


図2 MNIST データを用いた場合の学習曲線

した class を定義し、mnist_main.py に実行コードを実装した。このとき、MNIST のデータは $28 \times 28 = 784$ ピクセルのデータからなり、状態は $784 \times 5 = 3920$ 次元となる。具体的なコードは添付のプログラムを参照されたい。

一回の試行における学習曲線を図2に示す。明らかに図1と比較して明らかに累積報酬の分散が大きく学習が進んでいないことが確認できる。要因として状態の次元が大きすぎることから、状態からソーティングの法則を学習できていないことが問題であると考えられる。また、スカラーのソーティングの場合は、各状態から決定的な最適方策が定まるのに対し、MNIST のベクトルによるソーティングは状態のうちのある要素から決定的に決まる訳ではなく、確率的な方策が必要であることも学習が難しい問題の一つであると考えられる。

このような問題に対して、階層方策の構築が有効であると考えられる。具体的には、方策を上層と下層で分離し、上層では5つの画像を5つのスカラー値に変換する方策を学習し、下層ではスカラー値からソーティングの方策を学習する。このようにすることで探索空間を分割でき、学習が容易になると考えられる。また、画像からスカラー値に変換する問題は環境とのインタラクションがないため、単純な回帰問題に落とし込むことができる。

上記提案を実装し、実行した結果を図3に示す。画像からスカラー値への変換は事前に学習し、その方策を用いてソートタスクを行う方策を学習した。検証の結果、思うように性能が向上しなかった。問題はいくつか存在するが、大きな問題は上層の識別の間違いを下層の DQN によって十分吸収できなかったことである。ただしこれは DQN の問題というより、上層が 784 次元を 1 次元に圧縮しているため十分な情報量が下層へ送り出せなかったことが問題であると考えられる。このこと

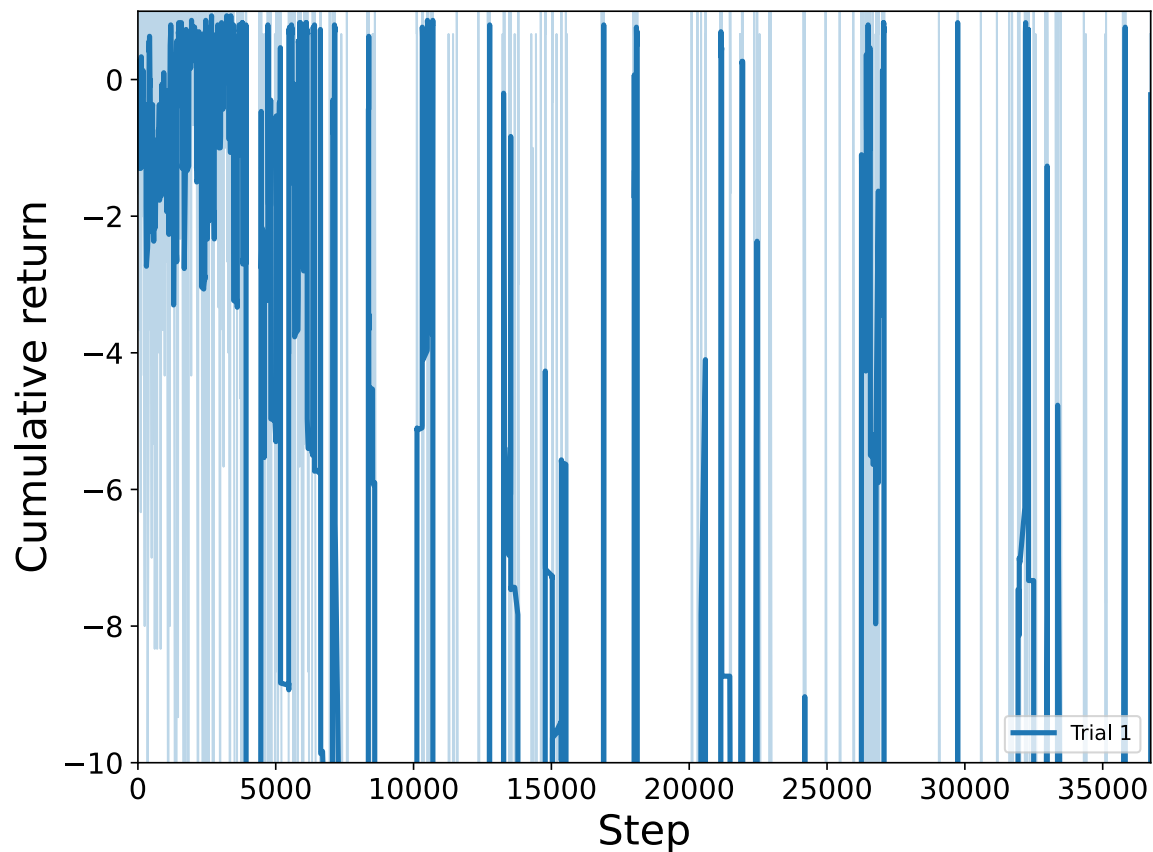


図3 Caption

から上層の学習において、もう少し大きな潜在表現に落とし込むことでより多くの情報を使う事が出来ると考えられる。この際に学習はエンコーダー・デコーダー型のアーキテクチャを用いることで実現できると思われる。

6 授業の感想

論理生命学では強化学習の基礎や理論的な内容を詳しく説明してくださり、ありがとうございました。私自身、ロボットラーニングが専門ではありますが、しっかりとした理論を学ぶ機会がなくとても参考になりました。私は Sutton の本で強化学習を勉強しましたが、石井先生の説明の進め方は Sutton のそれとは違い、新たな視点を得ることが出来ました。特に Actor-Critic の説明や LSTD の詳細な説明など、なぜそうなるのかというところが以前よりクリアになったと感じています。また、Bellman 作用素の説明も詳しく、わかりやすい説明で非常に助かりました。本講義の内容を今後の研究に活かしていきたいと思います。

プログラムリスト

- env.py: 通常のソーティング, MNIST, 階層における環境クラス
- agent.py: 通常及び階層における DQN, MNIST における DQN のエージェントクラス
- replaybuffer.py: replaybuffer クラス

- `main.py`: 通常のソーティング学習実行ファイル
- `mnist_main.py`: MNIST のソーティング学習実行ファイル
- `hierarchy_main.py`: 階層 MNIST のソーティング学習実行ファイル